CCSU BCA | SEMESTER I

Problem Solving Techniques

Algorithms, flowcharts, C programming and structured practice

COURSE CODE	BCA-1004T | Archive label: 19010
SEMESTER	BCA Semester I
SYLLABUS	CCSU BCA 2025-26 / NEP

This guide moves from understanding a problem to designing an algorithm, coding it in C, testing it and explaining the result. Practice by hand as well as on a compiler.

Targeting 80+ is a study goal, not a guaranteed result. Use these notes with class material, the current syllabus and timed paper practice.

How to use this guide

Learn the short unit summary first. Then answer each question without looking, compare your answer with the model, and rewrite it in your own words. For calculations and programs, write every step and test with a small example.

Contents

- Unit I | Problem-solving and algorithm development
- Unit II | Structured programming and control
- Unit III | Number problems and statistics
- Unit IV | Modular programming, recursion and arrays
- Question-paper coverage and pattern
- Last-five-year archive check
- Learning resources and references

Unit I | Problem-solving and algorithm development

A reliable solution begins with a precise problem statement. Identify valid inputs, expected outputs, constraints and special cases before writing steps.

Core ideas to remember

- Separate a problem from a particular instance: the problem is general; the instance supplies actual input values.
- A sound algorithm is finite, unambiguous, correct for valid inputs and practical in time and space.
- Write input/output specifications, validate input, and state preconditions and postconditions.
- Break a large task into subproblems, solve them, then review the result with test cases.
- Efficiency concerns resources such as execution time and memory; correctness comes first.

Important questions with model answers

Question: What is an algorithm? List its main characteristics.

Model answer

An algorithm is a finite, ordered set of unambiguous steps that transforms specified input into the required output. It should terminate, be precise enough to execute, accept valid input, produce the correct result and use feasible resources. For example, to find the larger of two numbers: read a and b; if a>b output a, otherwise output b. State what happens when a=b if the problem needs a distinct tie rule.

Question: Explain the problem-solving cycle used before coding.

Model answer

First restate the problem in your own words. Identify input, output, constraints and unusual cases. Plan a method and split it into smaller tasks. Trace the method on a small example and check that it covers boundary cases. Write pseudocode or a flowchart, implement one part at a time, compile and test, then review correctness and clarity. This cycle catches mistaken assumptions before they become code defects.

Unit II | Structured programming and control

Structured programming builds a program from sequence, selection and repetition. Good answers show the condition or loop boundary and explain why it terminates.

Core ideas to remember

- Sequence runs statements in order; selection chooses a branch; repetition repeats a block while a rule holds.
- Use if/else for a two-way choice and switch for a discrete choice among cases.
- A while loop checks before its body; a do-while loop runs the body at least once.
- A sentinel is a special input value that signals the end of data, such as -1.
- Translate pseudocode into C in small pieces and test the condition at zero, one and boundary values.

Important questions with model answers

Question: Compare while and do-while loops.

Model answer

A while loop is entry-controlled: its condition is tested before each pass, so its body may run zero times. A do-while loop is exit-controlled: the body runs once before the condition is tested, so it runs at least once. Use while when the loop may not need to start, such as reading until an input is available. Use do-while for a menu that must be shown at least once.

Question: Write the logic for displaying odd numbers from 1 through n.

Model answer

Initialize i to 1. While i is less than or equal to n, display i and add 2 to i. The loop visits 1, 3, 5 and so on, so every displayed value is odd and no value exceeds n. If n is less than 1, the loop displays nothing. In C, the loop condition is i <= n and the update is i += 2.

Unit III | Number problems and statistics

Digit problems use division and remainder. Statistical summaries use a small number of running variables, often while reading sentinel-controlled input.

Core ideas to remember

- For a positive integer n, the last digit is n % 10 and removing it is n / 10 using integer division.
- A palindrome reads the same in reverse. A three-digit Armstrong number equals the sum of the cubes of its digits.
- A prime number greater than 1 has exactly two positive divisors; test divisors only through the square root.
- Factorial is n! = n(n-1)...1, with 0! = 1.
- For a sequence, keep count, sum, minimum and maximum; compute average as sum/count after confirming count is non-zero.

C examples: factorial and Armstrong check

```
#include <stdio.h>

int main(void) {
    int n, digit, original, sum = 0, factorial = 1;
    int i;
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        factorial *= i;
    }

    original = n;
    while (n > 0) {
        digit = n % 10;
        sum += digit * digit * digit;
        n /= 10;
    }

    printf("Factorial: %d\n", factorial);
    if (sum == original) {
        printf("Armstrong number\n");
    } else {
        printf("Not an Armstrong number\n");
    }
    return 0;
}
```

Important questions with model answers

Question: How does a digit-reversal algorithm work?

Model answer

Save the original number if it is needed later. Set reverse to zero. While n is greater than zero, take digit=n%10, update reverse=reverse*10+digit, then set n=n/10 using integer division. For example, 582 becomes 2, then 28, then 285. For inputs that can be negative, decide whether to preserve the sign. For inputs ending in zero, the numerical reverse drops leading zeros, which is usually correct for integer output.

Question: What is sentinel-controlled repetition? Give a safe average calculation.

Model answer

A sentinel is a value that ends input and is not included in the data. Set count=0 and sum=0. Read a value; while it is not the sentinel, add it to sum, increase count and read the next value. Before calculating average, check count>0; otherwise division by zero would occur. A sentinel must be chosen so it cannot be mistaken for a valid data item, or the input format must provide another way to indicate the end.

Unit IV | Modular programming, recursion and arrays

Functions organize a solution into named tasks. Arrays store multiple values of one type. Recursion solves a problem by reducing it to a smaller instance with a base case.

Core ideas to remember

- A function prototype declares its name, return type and parameters before use; a definition supplies the body.
- A recursive function needs a base case and a step that moves toward it.
- Arrays use zero-based indexes in C; for an array of n elements, valid indexes are 0 through n-1.
- Binary search requires sorted data; selection sort repeatedly places the smallest remaining value.
- A two-dimensional array can represent a matrix; matrix addition requires equal dimensions.

C example: add two 2 x 2 matrices

```
#include <stdio.h>

int main(void) {
    int a[2][2], b[2][2], c[2][2];
    int row, col;

    for (row = 0; row < 2; row++) {
        for (col = 0; col < 2; col++) scanf("%d", &a[row][col]);
    }
    for (row = 0; row < 2; row++) {
        for (col = 0; col < 2; col++) scanf("%d", &b[row][col]);
    }
    for (row = 0; row < 2; row++) {
        for (col = 0; col < 2; col++) {
            c[row][col] = a[row][col] + b[row][col];
            printf("%d ", c[row][col]);
        }
        printf("\n");
    }
    return 0;
}
```

Important questions with model answers

Question: Explain recursion using factorial.

Model answer

Factorial follows n! = n x (n-1)! for n>0, with the base case 0!=1. A recursive function returns 1 when n is 0; otherwise it returns n multiplied by factorial(n-1). The argument decreases on every call, so it eventually reaches zero. Without a correct base case, calls continue until the program exhausts its call stack.

Question: When may binary search be used, and what is its main idea?

Model answer

Binary search requires the array to be sorted. Compare the target with the middle element. If they match, stop. If the target is smaller, continue in the left half; if larger, continue in the right half. Repeat until found or the search interval is empty. Each comparison roughly halves the remaining interval, so its time grows logarithmically with the number of elements. It is not valid on unsorted input.

Paper pattern and topic analysis

The December 2025 paper transcription reports a 3-hour, 75-mark external paper using 5 questions x 3 marks, any 2 of 3 x 7.5 marks, and any 3 of 5 x 15 marks. The syllabus assigns 25 internal and 75 external marks to the theory paper; practical assessment is separate.

The 2025 transcription includes algorithm definition, if/else, C program structure, loops, Armstrong number, factorial and 2x2 matrix operations. This points to short definitions plus longer code-writing and comparison questions. The 2021-2024 archive papers use the older title Programming Principles and Algorithm; examples such as flowcharts, functions and C basics overlap, but the old syllabus is not identical.

What to prioritize

- Write and trace algorithms before typing code; test normal, boundary and invalid inputs.
- Practise selection and loops, including while versus do-while and nested loops.
- Prepare complete programs for factorial, Armstrong, palindrome, prime, array statistics and matrix operations.
- Explain functions, recursion, arrays, debugging and code testing in short, structured paragraphs.

Last-five-year archive check

The CCSUStudy archive lists the current Problem Solving Techniques paper for December 2025 (archive label 19010). It also lists earlier Programming Principles and Algorithm papers for 2021, 2022, 2023 and 2024, covering the five-calendar-year window with a changed course title and syllabus. The December 2024 legacy paper includes C keywords, preprocessor directives, break/exit, functions and flowcharts. Treat those as partial overlap, not as exact copies of the current paper.

How to read the evidence

The current syllabus and the archive's paper-year list are checked. The 2025 pattern and question themes are taken from a third-party transcription. The 2024 legacy paper provides examples of older overlap. Recurrence across all five papers is not claimed.

Learning resources and source list

The paper archive is a discovery aid, not the university's official syllabus. The current course titles and units below follow the 2025-26 CCSU syllabus reproduced by an affiliated college. Older paper titles are identified as legacy or related material wherever they differ.

CCSU BCA syllabus and unit list (2025-26 reproduction)

CCSUStudy BCA question-paper archive

December 2025 Problem Solving Techniques paper listing

December 2024 legacy Programming Principles and Algorithm paper

NPTEL Introduction to Programming in C, IIT Kanpur

NPTEL Problem Solving through Programming in C, IIT Kharagpur

Coverage note: no question bank can predict the exact paper. Treat the priority list as a revision map and prepare every listed unit.